

The Composition Root — Builder & CliBuilder

Where every part of the node is constructed and wired together into one HathorManager — the Builder pattern at the scale of a whole full node.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 24 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Composition root · Builder pattern · Dependency injection · CliBuilder · Builder · ResourcesBuilder · Wiring · HathorManager

Table of Contents

Chapter 24 — The Composition Root: Builder & CliBuilder	3
24.1 Localization	3
24.2 What it does, and why it exists	4
Why centralize the wiring?	5
Why it makes the node testable	5
24.3 The concepts it rests on	5
24.4 The code, walked	6
24.4.1 A generic builder, first	6
24.4.2 The real Builder — tests and simulator	7
24.4.3 The real CliBuilder — production	10
24.4.4 Two composition roots, one job — and the debt	13
24.4.5 ResourcesBuilder, briefly	13
24.5 How it plugs into the lifecycle	14
Recap	15
Footnotes	16

Chapter 24 — The Composition Root: Builder & CliBuilder

What you'll learn in this chapter

- What a composition root is, and why a program should have exactly one place where its objects are constructed and connected.
- How `hathor-core` builds a full node: storage, indexes, pub-sub, the event manager, consensus, verification, P2P, and the `HathorManager` that ties them together.
- The two composition roots that coexist in the codebase — `Builder` (for tests and the simulator) and `CliBuilder` (for production) — and why there are two.
- Where this "builder phase" sits in the life of a node: between the reactor being readied (Ch. 23) and the manager being started (Ch. 29).
- The honest trade-off of having two builders that can drift apart, and the `TODD` in the code that admits it.

This chapter is step 5 of the life-of-a-node story from Chapter 0 (§0.3, "the node is assembled — the builder phase"). Everything Part II has covered so far — the command line (Ch. 21), the settings (Ch. 22), the reactor (Ch. 23) — has been preparation. This is where those ingredients are turned into a real, wired-together node object, ready to be switched on.

24.1 Localization

The package `hathor/builder/` is small in file count but central in role: nothing else can produce a working node without it. There is also a second builder, for the command line, that lives next door under `hathor_cli/`.

```

hathor-core/
├── hathor/
│   ├── builder/
│   │   ├── __init__.py
│   │   ├── builder.py
│   │   ├── resources_builder.py
│   │   └── sysctl_builder.py
│   ├── manager.py
│   ├── consensus/
│   ├── transaction/storage/
│   ├── indexes/
│   ├── p2p/
│   └── verification/
└── hathor_cli/
    ├── builder.py
    └── run_node.py

```

- ← YOU ARE HERE (part 1)
- ← exports Builder, BuildArtifacts, ResourcesBuilder, SyncSupportLevel
- ← Builder (tests + simulator path)
- ← ResourcesBuilder (the HTTP/REST API tree)
- ← builds the sysctl control surface (Ch. 41)
- ← HathorManager – the object being built (Ch. 29)
- ← ConsensusAlgorithm (Ch. 32)
- ← tx storage backends (Ch. 27)
- ← IndexesManager (Ch. 28)
- ← ConnectionsManager (Ch. 34)
- ← VerificationService (Ch. 31)
- ← YOU ARE HERE (part 2): CliBuilder
- ← thin shim; imports RunNode from hathor_cli

Two things to notice. First, the two builders live in different top-level packages: `Builder` is in `hathor/builder/`, but `CliBuilder` is in `hathor_cli/builder.py` (`hathor_cli/builder.py:60`). Second, the class that actually drives the production boot — `RunNode` — lives in `hathor_cli/run_node.py`, and it is the one that imports and uses `CliBuilder` (`hathor_cli/run_node.py:207`). This is the same run-node body Chapter 21 walked as "the CLI surface": there, the focus was argument parsing and the boot sequence; here, the focus is the one step inside it that assembles the node.

Context. This package is the assembly line of the node. Every other chapter in Part II describes a part — the vertex model, consensus, the P2P manager, the indexes. This chapter describes the one place where all those parts are instantiated, handed their dependencies, and bolted onto a single `HathorManager`. If you want to know "what gets built, in what order, with what wired to what," these two files are where you look.

24.2 What it does, and why it exists

A running node is not one object. It is roughly a dozen long-lived objects that hold references to each other: a transaction store, a set of indexes over that store, a pub-sub bus, an event manager that listens on the bus, a consensus engine, a verification service, a P2P connection manager, a wallet, a mining service. Each has its own constructor arguments — and many of those arguments are other objects from the same list. Consensus needs the pub-sub bus. The event manager needs the bus and the reactor. The verification service needs the settings, the difficulty algorithm, and the feature service.

Somewhere, someone has to call all those constructors **in the right order**, passing each object the collaborators it depends on. That "somewhere" is the **composition root**¹.

Why centralize the wiring?

Consider the alternative: each entry point — the test suite, the simulator, the production CLI — constructs the node inline, where it happens to need one. You would then have the same dozen-line wiring incantation copied in three places. Change one dependency — say, consensus now needs a new collaborator — and you must find and fix every copy. Miss one, and that code path silently builds a subtly different, broken node.

Centralizing the wiring means the construction order and the dependency edges live in **one** place. Every caller gets the same node, built the same way. When a component's constructor changes, there is exactly one site to update.

Why it makes the node testable

A composition root also makes dependencies explicit and injectable — the heart of the dependency-injection idea from Chapter 3. Because the manager receives its collaborators as constructor arguments instead of creating them itself, a test can hand it a fake. The manager does not know, and does not care, whether its storage writes to RocksDB or to a Python dictionary in memory. It talks to the abstract `TransactionStorage` interface. That substitutability is what makes the whole node testable, and it is decided here, in the builder.

So the builder package exists to answer one question, once: given a configuration, produce a fully wired `HathorManager`.

24.3 The concepts it rests on

This chapter pays off ideas the foundations already taught. Recap, don't re-teach.

Recap — the Builder pattern (full treatment in Ch. 3 §3.3). A Builder separates how an object is assembled from what it finally is. You push configuration in, one call at a time, then ask for the finished product. The builder holds the half-built state; you hold only the handle. Hathor's `Builder` is the textbook shape: fluent `set_*` methods that each return the builder, and a terminal `build()` that produces the result. → full treatment in Ch. 3.

Recap — dependency injection (full treatment in Ch. 3). Objects receive their collaborators from outside rather than constructing them internally. The builder is where the injection happens: it creates the pub-sub bus once, then passes that one bus into consensus, into the event manager, into the P2P manager. None of those build their own bus. → full treatment in Ch. 3.

Recap — Facade (full treatment in Ch. 3 §3.6 and Ch. 29). `HathorManager` is the Facade: one object presenting a small interface (`start()` , `stop()` , `on_new_tx()`) over a complex subsystem. The builder's job is to construct that subsystem and place the facade on top of it. The builder builds the thing; Ch. 29 describes what the thing does. → full treatment in Ch. 29.

Recap — interchangeable backends (full treatment in Ch. 1 and Ch. 27). The same node can keep its ledger in memory (for tests) or in RocksDB (for production). That choice is made here, in the builder, by instantiating one storage class or the other. The rest of the node speaks only to the abstract `TransactionStorage` interface and never learns which backend it got. → full treatment in Ch. 27.

24.4 The code, walked

24.4.1 A generic builder, first

Before the real node, a toy. Strip the Builder pattern down to its bones: a thing assembled from optional parts, produced by one terminal call.

```
class SandwichBuilder:
    def __init__(self):
        self._bread = None
        self._filling = None

    def set_bread(self, bread):
        self._bread = bread
        return self          # fluent: return self so the calls chain

    def set_filling(self, filling):
        self._filling = filling
        return self

    def build(self):
        bread = self._bread or "white"          # a default, if unset
        return Sandwich(bread, self._filling)

# usage
sandwich = SandwichBuilder().set_bread("rye").set_filling("cheese").build()
```

Three ideas to carry forward:

1. **Setters return `self`**. That is the fluent interface — it lets calls chain in one line. Each setter only stashes a value.
2. **`build()` is terminal**. Construction happens at the end, once, when all the configuration is in.
3. **Defaults live in the builder**. If you never call `set_bread`, the builder picks one. The caller need not specify everything.

Hathor's `Builder` is this exact shape, scaled up to a full node, with one addition: instead of computing defaults inline inside `build()`, it uses lazy, memoized getters. We'll see why that matters.

24.4.2 The real `Builder` — tests and simulator

`Builder` lives at `hathor/builder/builder.py:118`. Its `__init__` does almost nothing but declare a long list of `Optional` slots, all initialized to `None` (`builder.py:127 – 201`). Each slot is a place a `set_*` call can drop a value, or a place a getter can later cache one:

```
# hathor/builder/builder.py (abridged from __init__, lines 127–166)
class Builder:
    def __init__(self) -> None:
        self.artifacts: Optional[BuildArtifacts] = None # the finished bundle
        self._settings: Optional[HathorSettingsType] = None
        self._reactor: Optional[Reactor] = None
        self._pubsub: Optional[PubSubManager] = None
        self._tx_storage: Optional[TransactionStorage] = None
        self._indexes_manager: Optional[IndexesManager] = None
        self._rocksdb_storage: Optional[RocksDBStorage] = None
        # ... a dozen more slots: wallet, consensus, feature_service, daa, etc.
```

The fluent setters. Each `set_*` checks that the node has not already been built, stashes a value, and returns `self`. The guard is a one-liner, `check_if_can_modify` (`builder.py:313`):

```
# hathor/builder/builder.py:313–320
def check_if_can_modify(self) -> None:
    if self.artifacts is not None:
        raise ValueError('cannot modify after build() is called')

def set_event_manager(self, event_manager: EventManager) -> 'Builder':
    self.check_if_can_modify()
    self._event_manager = event_manager
    return self
```

That guard enforces an invariant: once you call `build()`, the builder freezes. You cannot retroactively swap a dependency on a node that already exists. (How does it know it has built? `build()` sets `self.artifacts`; the guard checks for it.)

The lazy, memoized getters. This is where Hathor's `Builder` departs from the sandwich toy. Instead of computing defaults inside `build()`, each component has a private `_get_or_create_*` method that returns the slot if it is set, otherwise constructs a default and caches it in the slot:

```
# hathor/builder/builder.py:444-447
def _get_or_create_pubsub(self) -> PubSubManager:
    if self._pubsub is None:
        self._pubsub = PubSubManager(self._get_reactor())
    return self._pubsub
```

Why cache? Because these objects are shared. The pub-sub bus that consensus receives must be the **same** bus the event manager listens on. If each getter built a fresh object on every call, consensus and the event manager would end up talking past each other on two different buses. Caching in the slot guarantees a single instance, no matter how many components ask for it. The pattern is create-once, then return-the-same-one. (This is the runtime cousin of the Singleton idea from Chapter 3, but scoped to one builder rather than the whole program — much safer.)

You can see the sharing in `_get_or_create_p2p_manager`, which pulls several already-memoized collaborators (`builder.py:463 - 481`):

```
# hathor/builder/builder.py:471-481 (abridged)
self._p2p_manager = ConnectionsManager(
    settings=self._get_or_create_settings(),
    reactor=reactor,
    my_peer=my_peer,
    pubsub=self._get_or_create_pubsub(),  # ← the one shared bus
    ssl=enable_ssl,
    whitelist_only=False,
    rng=self._rng,
    ...
)
```

The `pubsub` here is the very same object every other getter receives. That is dependency injection (Ch. 3) made concrete.

The backend choice. The memory-versus-RocksDB decision (Ch. 27) is made by which getter path runs. In `Builder`, the tx storage getter always builds a RocksDB-backed store, but the RocksDB storage itself can be a temporary directory when no path was set — which is how the simulator and tests get a throwaway store (`builder.py:455 - 461`):

```
# hathor/builder/builder.py:455-461
def _get_or_create_rocksdb_storage(self) -> RocksDBStorage:
    if self._rocksdb_storage is None:
        self._rocksdb_storage = RocksDBStorage(
            path=self._rocksdb_path,
            cache_capacity=self._rocksdb_cache_capacity,
        ) if self._rocksdb_path else RocksDBStorage.create_temp(self._rocksdb_cache_capacity)
    return self._rocksdb_storage
```


The rest of the node sees only the abstract `TransactionStorage` type (`builder.py:500`) and is none the wiser about where the bytes land.

build() — the orchestration. `build()` (`builder.py:203`) is the terminal step. It first asserts you have not built twice and that a sync version is enabled (`builder.py:204 – 208`), then calls the getters in dependency order and hands every result to the `HathorManager` constructor:

```
# hathor/builder/builder.py:210–235 (abridged)
settings = self._get_or_create_settings()
reactor = self._get_reactor()
pubsub = self._get_or_create_pubsub()
peer = self._get_peer()
consensus_algorithm = self._get_or_create_consensus()
p2p_manager = self._get_or_create_p2p_manager()
wallet = self._get_or_create_wallet()
event_manager = self._get_or_create_event_manager()
indexes = self._get_or_create_indexes_manager()
tx_storage = self._get_or_create_tx_storage()
feature_service = self._get_or_create_feature_service()
verification_service = self._get_or_create_verification_service()
daa = self._get_or_create_daa()
# ... and several more
```

The single line where the facade is born and every dependency is injected is the `HathorManager(...)` call (`builder.py:257`):

```
# hathor/builder/builder.py:257–283 (abridged)
manager = HathorManager(
    reactor,
    settings=settings,
    pubsub=pubsub,
    consensus_algorithm=consensus_algorithm,
    daa=daa,
    peer=peer,
    tx_storage=tx_storage,
    p2p_manager=p2p_manager,
    event_manager=event_manager,
    wallet=wallet,
    bit_signaling_service=bit_signaling_service,
    verification_service=verification_service,
    cpu_mining_service=cpu_mining_service,
    feature_service=feature_service,
    # ... ~20 collaborators in total
)
```

BuildArtifacts — handing back the parts, not only the whole. `build()` does not return the bare manager. It returns a `BuildArtifacts`, a `NamedTuple` (`builder.py:93`) that bundles the manager together with its collaborators:

```
# hathor/builder/builder.py:93-109 (abridged)
class BuildArtifacts(NamedTuple):
    """Artifacts created by a builder."""
    peer: PrivatePeer
    settings: HathorSettingsType
    reactor: Reactor
    manager: HathorManager
    p2p_manager: ConnectionsManager
    pubsub: PubSubManager
    consensus: ConsensusAlgorithm
    tx_storage: TransactionStorage
    indexes: IndexesManager
    wallet: Optional[BaseWallet]
    # ...
```

Why hand back the parts? Tests and the simulator routinely need to reach past the facade — to inspect the storage directly, push a transaction onto the pub-sub bus, or assert on the indexes. `BuildArtifacts` gives them a handle to each piece without forcing them to re-derive it from the manager. The `build()` method stores the bundle in `self.artifacts` and returns it (`builder.py:293 – 311`); storing it is also what arms the `check_if_can_modify` freeze.

24.4.3 The real `CliBuilder` — production

`CliBuilder` lives at `hathor_cli/builder.py:60`. It is the path the production command line drives. It does the same job as `Builder` — produce a wired `HathorManager` — but its shape is different. There are no fluent `set_*` setters and no `BuildArtifacts` bundle. Instead, one large method, `create_manager(reactor)`, reads everything it needs from the parsed CLI arguments (`self._args`) and constructs the node top to bottom.

The class opens with an honest admission (`hathor_cli/builder.py:60 – 64`):

```
# hathor_cli/builder.py:60-64
class CliBuilder:
    """CliBuilder builds the core objects from args.

    TODO Refactor to use Builder. It could even be ported to a Builder.from_args classmethod.
    """
```

`create_manager` (`hathor_cli/builder.py:79`) is a long, linear procedure. Walked in order, it builds:

1. **Settings** — `get_global_settings()` (`hathor_cli/builder.py:93`). The settings object (Ch. 22) is read once and threaded into nearly every component below.
2. **Peer identity** — `PrivatePeer.create_from_json_path(...)` if a peer file was given, else `PrivatePeer.auto_generated()` (`hathor_cli/builder.py:102 – 105`). This is the node's cryptographic identity on the P2P network (Ch. 34).
3. **Transaction storage and indexes** — production is **always RocksDB**. The old `--memory-storage` and `--memory-indexes` flags are now deprecated and

rejected (hathor_cli/builder.py:127 – 128); the builder then opens the RocksDB store and an index manager over it (builder.py:130 – 160):

```
# hathor_cli/builder.py:127–158 (abridged)
memory_msg = 'is deprecated. use --temp-data instead'
self.check_or_raise(not self._args.memory_storage, f'--memory-storage {memory_msg}')
self.check_or_raise(not self._args.memory_indexes, f'--memory-indexes {memory_msg}')

self.check_or_raise(bool(self._args.data) or self._args.temp_data,
                    'either --data or --temp-data is expected')
self.rocksdb_storage = (
    RocksDBStorage(path=self._args.data, cache_capacity=self._args.rocksdb_cache)
    if self._args.data else RocksDBStorage.create_temp(self._args.rocksdb_cache)
)
indexes = RocksDBIndexesManager(self.rocksdb_storage, settings=settings)
tx_storage = TransactionRocksDBStorage(
    reactor=reactor, rocksdb_storage=self.rocksdb_storage,
    settings=settings, indexes=indexes, ...)
```

A throwaway store is still possible — `--temp-data` routes to `RocksDBStorage.create_temp(...)` (builder.py:134) — but it is still RocksDB, just in a temporary directory. `check_or_raise` (hathor_cli/builder.py:69) is `CliBuilder`'s way of rejecting a misconfigured launch with a clean message (a `BuilderError`) rather than a raw traceback.

4. **Event storage, feature storage** — `EventRocksDBStorage(...)` and `FeatureActivationStorage(...)` (hathor_cli/builder.py:161 – 162).

5. **Wallet and hostname** — `self.create_wallet()` if a wallet was configured (hathor_cli/builder.py:181 – 182) and `self.get_hostname()` (:185).

6. **Pub-sub bus** — `PubSubManager(reactor)` (hathor_cli/builder.py:195). Created once, injected everywhere below.

7. **Event manager** — `EventManager(...)` (hathor_cli/builder.py:210), given the event storage, the optional WebSocket factory, the reactor, and the same pub-sub bus.

8. **Feature service** — `FeatureService(...)` (hathor_cli/builder.py:246), then later the **bit-signaling service** `BitSignalingService(...)` (:265). Together these drive feature activation, the node's on-chain upgrade mechanism (Ch. 38).

9. **Consensus** — `ConsensusAlgorithm(...)` (hathor_cli/builder.py:249), given the soft-voided tx ids, settings, and feature service (Ch. 32).

10. **Difficulty algorithm (DAA)** — `DifficultyAdjustmentAlgorithm(...)` (hathor_cli/builder.py:280), Ch. 32.

11. **Vertex verifiers and verification service** — `VertexVerifiers.create_defaults(...)` then `VerificationService(...)` (hathor_cli/builder.py:282 – 294). These enforce every protocol rule on incoming vertices (Ch. 31).

12. **CPU mining service** — `CpuMiningService()` (hathor_cli/builder.py:296).

13. **P2P manager** — `ConnectionsManager(...)` (hathor_cli/builder.py:298), given the settings, reactor, peer, and the same pub-sub bus:

```
# hathor_cli/builder.py:298-308 (abridged)
p2p_manager = ConnectionsManager(
    settings=settings,
    reactor=reactor,
    my_peer=peer,
    pubsub=pubsub,          # ← the one shared bus, again
    ssl=True,
    whitelist_only=False,
    rng=Random(),
    ...
)
```

1. **Vertex handler** — `VertexHandler(...)` (`hathor_cli/builder.py:310`), the ingestion pipeline that ties verification, consensus, and execution together (Ch. 33).
2. **The manager** — `HathorManager(...)` (`hathor_cli/builder.py:338`), the facade, receiving every object built above:

```
# hathor_cli/builder.py:338-363 (abridged)
self.manager = HathorManager(
    reactor,
    settings=settings,
    hostname=hostname,
    pubsub=pubsub,
    consensus_algorithm=consensus_algorithm,
    daa=daa,
    peer=peer,
    tx_storage=tx_storage,
    p2p_manager=p2p_manager,
    event_manager=event_manager,
    wallet=self.wallet,
    checkpoints=settings.CHECKPOINTS,
    bit_signaling_service=bit_signaling_service,
    verification_service=verification_service,
    cpu_mining_service=cpu_mining_service,
    vertex_handler=vertex_handler,
    feature_service=self.feature_service,
    # ... ~20 collaborators in total
)
```

16. Post-construction wiring, then return. After the manager exists, the builder wires it back into its collaborators (`p2p_manager.set_manager(self.manager)`, `hathor_cli/builder.py:370`), optionally attaches a Stratum mining factory (`:374 - 377`), registers peer discovery and `--listen` addresses (`:388 - 403`), and finally returns the bare manager (`hathor_cli/builder.py:409`) — no `BuildArtifacts` bundle. Production code does not need to reach past the facade the way tests do.

Compare the two `HathorManager(...)` calls — `builder.py:257` and `hathor_cli/builder.py:338`. They pass nearly the same arguments in nearly the same shape. That near-duplication is the whole point of the next section.

24.4.4 Two composition roots, one job — and the debt

The codebase has **two** composition roots that build the same product:

	Builder	CliBuilder
File	<code>hathor/builder/builder.py</code>	<code>hathor_cli/builder.py</code>
Configured by	fluent <code>set_*</code> calls	parsed CLI args (<code>self._args</code>)
Returns	<code>BuildArtifacts</code> (manager + parts)	the bare <code>HathorManager</code>
Internal style	lazy, memoized <code>_get_or_create_*</code>	one linear <code>create_manager</code>
Used by	tests, simulator	<code>run_node</code> (production)

Why two? Historically, the test/simulator `Builder` grew up as a clean, composable API; the production path accreted its own construction code, tied to argument parsing, in `CliBuilder`. They were never unified. The `TODO` at `hathor_cli/builder.py:63` is the project acknowledging it in its own words: "Refactor to use Builder. It could even be ported to a `Builder.from_args` classmethod." — i.e. `CliBuilder` should eventually delegate to `Builder` instead of duplicating its wiring.

The cost is real and worth stating plainly. The same wiring — which collaborator goes into which constructor — is written twice. When consensus gains a new dependency, both `builder.py:257` and `hathor_cli/builder.py:338` must change in lockstep. Miss one and the two paths build different nodes: tests pass against a node that production never actually runs. This is precisely the drift a single composition root exists to prevent — and here the project has two of them. The duplication in the two `HathorManager(...)` calls above is the visible symptom.

So, a practical rule for a junior contributor: **when you change how a component is constructed, search both files**. Treat `CliBuilder` as authoritative for what production runs, and `Builder` as authoritative for what tests and the simulator run. They are supposed to agree, but the language does not enforce it — only discipline does.

24.4.5 ResourcesBuilder, briefly

The third export in `hathor/builder/__init__.py` (`__init__.py:16`), `ResourcesBuilder` (`hathor/builder/resources_builder.py:66`), is not a composition root for the node. It is a smaller builder that takes an already-built manager plus a couple of its pieces (the wallet, the event WebSocket factory, the feature service) — see its `__init__` (`resources_builder.py:67 – 83`) — and assembles the node's HTTP/REST resource tree,

the API surface the node exposes over the wire (`build()` at `resources_builder.py:85`, `create_resources()` at `:114`). It runs after the manager is built. It is mentioned here only so you know what the fourth name in the builder package is; its detail belongs with the API chapters.

24.5 How it plugs into the lifecycle

This chapter is step 5 of Act I's boot sequence (Ch. 0 §0.3). It sits between two neighbors:

- **Before it:** the reactor is selected and readied (Ch. 23), the settings are loaded (Ch. 22), and the CLI has parsed its arguments (Ch. 21). The builder consumes all three: it takes the reactor as the argument to `create_manager`, reads the settings via `get_global_settings()`, and reads the CLI flags from `self._args`.
- **After it:** the manager goes live (Ch. 29). The builder returns a fully wired but not-yet-started `HathorManager`. Nothing is listening, no peers are connected, no consensus is running. That happens only when the caller calls `manager.start()`.

You can see the hand-off in the production run-node body, inside `RunNode.prepare` (`hathor_cli/run_node.py:207 – 216`):

```
# hathor_cli/run_node.py:205–216 (abridged)
from hathor.builder import ResourcesBuilder
from hathor.exception import BuilderError
from hathor_cli.builder import CliBuilder
builder = CliBuilder(self._args)
try:
    self.manager = builder.create_manager(reactor)  # ← the builder phase
except BuilderError as err:
    self.log.error(str(err))
    sys.exit(2)

self.tx_storage = self.manager.tx_storage
self.wallet = self.manager.wallet
```

`RunNode` constructs the `CliBuilder` with the parsed args, asks it for a manager, and then pulls a couple of references off the manager (the storage, the wallet) for its own convenience (`hathor_cli/run_node.py:215 – 216`). Note the `try/except`: a `BuilderError` raised by `check_or_raise` becomes a clean `sys.exit(2)` with a logged message, not a crash dump — that is the whole reason `check_or_raise` exists. The later call to `manager.start()` — the moment the node actually goes live — is the subject of Chapter 29.

Recap

Concern	Where	Central type
What a composition root is	this chapter, §24.2	— (a role, not a class)
Build a node for tests/simulator	<code>hathor/builder/builder.py:118</code>	<code>Builder</code>
Bundle the manager with its parts	<code>hathor/builder/builder.py:93</code>	<code>BuildArtifacts</code>
Freeze the builder after build	<code>hathor/builder/builder.py:313</code>	<code>check_if_can_modify</code>
Build a node for production	<code>hathor_cli/builder.py:79</code>	<code>CliBuilder.create_manager</code>
Reject a bad launch cleanly	<code>hathor_cli/builder.py:69</code>	<code>check_or_raise</code>
Drive the production boot	<code>hathor_cli/run_node.py:208</code>	<code>RunNode.prepare</code>
Build the HTTP/REST API tree	<code>hathor/builder/resources_builder.py:66</code>	<code>ResourcesBuilder</code>
The duplication debt	<code>hathor_cli/builder.py:63</code>	the <code>TODO</code>

A composition root is the one place an application builds its object graph and injects dependencies. `hathor-core` has two of them: `Builder` (fluent setters, memoized getters, returns a `BuildArtifacts` bundle) for tests and the simulator, and `CliBuilder` (one linear `create_manager` reading CLI args, returns the bare manager) for production. Both converge on a near-identical `HathorManager(...)` call that injects every collaborator — and that near-duplication is honest technical debt the code's own `TODO` admits. The builder phase consumes the reactor (Ch. 23), the settings (Ch. 22), and the CLI args (Ch. 21), and produces a wired-but-not-started manager that Chapter 29 will bring to life.

The next chapter steps into the data the node spends its life managing: the **vertex model** (Ch. 25) — transactions and blocks as nodes in a DAG. Once you know what a vertex is, the storage, verification, and consensus services this chapter wired together will finally have something concrete to act on.

Footnotes

1. A **composition root** is the single place in an application where the object graph is assembled — where you call the constructors and decide who depends on whom. The term comes from the dependency-injection literature. The rule of thumb: construct objects in one place, as close to the program's entry point as you can, and pass dependencies inward. Keep the rest of the code free of constructor calls for its own collaborators. ↩